Sistemas y Tecnologías de Administración del conocimiento

Jesús Clemente Chio Rodríguez

Activity 3

al07004708: Sebastián Soria González

al07026440: Gustavo Andres Ontiveros Ipiña

al03095697: Roberto Abisai Hernández De La Rosa

Table of Contents

1. Objective and Document Repository....3
2. Selected Approach: Path A (TF-IDF)....3
3. Execution....3
4. Test Results....3
5. Organizational Utility....4
6. Observed Limitations....4
7. Areas for Improvement....4
References....5
Appendix A. Execution Evidence....6

Top-K Document Recommender with Cosine Similarity

1. Objective and Document Repository

Given a document ID or a free-text query, the system recommends the **5 most similar documents** from an internal repository, together with their similarity scores. This lets employees quickly find the policy, guide, or past ticket that already solves their case.

The repository (`corpus.csv`) simulates the document base of "Altavista", a fictional IT services company. It contains **30 documents** in English (57 to 78 words each) with the columns `id`, `title`, `text`, and `category`, and no duplicated identifiers: Policy/Process (10), Support/Incidents (10), FAQ (5), and Minutes (5). The code, corpus, and evidence are available in the project repository (Andres145876, 2026).

2. Selected Approach: Path A (TF-IDF)

TF-IDF with cosine similarity, two classic information retrieval techniques (Salton & Buckley, 1988; Manning et al., 2008), was chosen for its simplicity and transparency: each score is explained by the terms two documents share. It was implemented with scikit-learn (Pedregosa et al., 2011). The script `recommender.py` follows five steps:

1. **Loading and validation** (`load_corpus`): checks that the file exists, that the required columns are present, and that there are no empty or duplicated IDs and no empty texts.
2. **Cleaning** (`clean_text`): lowercasing, removal of accents and special characters, English stopwords, and light stemming (e.g., "passwords" → "password").
3. **Vectorization**: `TfidfVectorizer` (with `sublinear_tf`) over the title + text of each document (scikit-learn developers, n.d.).
4. **Similarity**: `cosine_similarity` between the seed or the query and the whole corpus.
5. **Top-5**: sorts scores from highest to lowest and, when an ID is used, excludes the seed document.

The code is organized into functions and the `Recommender` class. It handles nonexistent IDs (suggesting similar IDs), empty queries or queries with no corpus vocabulary, missing files, and invalid `--k` values with clear messages.

3. Execution

It requires Python 3.9+ and `pip install -r requirements.txt` (scikit-learn and numpy). The command-line interface accepts a document ID (`python recommender.py --id DOC001`), free text (`--query "text"`), or an interactive menu (`python recommender.py`). The `--save` option stores the output in the `evidencias/` folder.

4. Test Results

Test	Input	Top result (score)	Coherent
1. Policy/Process	DOC001 – Password and MFA Policy	DOC021 – FAQ: How do I reset my password? (0.4134)	4 of 5
2. Support/Incidents	DOC012 – Ticket: VPN will not connect	DOC022 – FAQ: How do I connect to the VPN? (0.5002)	4 of 5
3. Free text	"My account is locked after I changed my password"	DOC011 – Ticket: User locked out (0.4455)	3 of 5

In all three tests, the top result is the most useful document for the input. Weaker results (positions 4 and 5) only share general words such as "account" or "ticket". The `--evaluate` metric yields 0.33 category agreement: the system groups documents by **topic**, not by document type, which is why a VPN ticket recommends the VPN policy and FAQ. Screenshots of each run are in Appendix A (Figures 1 to 4).

5. Organizational Utility

The recommender **reduces rework**: whoever handles the VPN ticket (Test 2) gets the FAQ and the policy that already solve it within seconds, instead of writing a solution from scratch. It also **speeds up decision-making**, because it gathers the rule (policy), the guide (FAQ), and past cases (tickets) in a single ranked list, making answers faster and more consistent.

6. Observed Limitations

- **Synonyms:** TF-IDF compares words, not meanings; "locked out" and "blocked" are not related unless they share terms.
- **Ambiguity:** the query "I can't log in after changing my password" retrieves DOC020 and DOC030 because of "log" (log files) and DOC004 and DOC028 because of "change" (change management).
- **Document length:** with 57–78 words per document, one or two shared generic words inflate the score, which explains the noise in the last positions.
- **Corpus bias:** topics with more documents (passwords, VPN) appear more often, and a topic with no documents is never recommended.

7. Areas for Improvement

- **Embeddings** (Path B, `sentence-transformers`; Reimers & Gurevych, 2019) or a hybrid approach to capture synonyms and reduce ambiguity.
- **Metadata** such as date, version, owner, and validity, to prioritize the current policy.
- **Role-based filters and security**, so each user only sees documents at their access level.
- **Continuous evaluation**: record whether the recommendation solved the case and track precision@5 over time; also, a minimum score threshold (e.g., 0.15) to hide weak results.

Project repository: https://github.com/Andres145876/document-recommender

Reflection video: https://youtu.be/LFop6uEI0LI?si=qHIQ2sbCKFXyJ_pq

References

Andres145876. (2026). *document-recommender* [Source code repository]. GitHub. https://github.com/Andres145876/document-recommender

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

scikit-learn developers. (n.d.). *Feature extraction* [User guide]. https://scikit-learn.org/stable/modules/feature_extraction.html

Appendix A. Execution Evidence

Console screenshots from running `recommender.py` with the corpus included in the repository. Each one shows the input, the Top-5 with its cosine similarity score, and the category of each document.

Figure 1

Test 1 (Policy/Process): recommendations for seed document DOC001

```
(.venv) C:\Users\Admin\Downloads\document-recommender\document-recommender>python recommender.py --id DOC001 --save evidencias/test1_policy_process.txt
================================================================================================
INPUT (ID): DOC001 | Policy/Process | Password and Multi-Factor Authentication Policy
================================================================================================
Top-5 recommended documents (cosine similarity)
#   ID       Score    Category            Title
------------------------------------------------------------------------------------------------
1   DOC021   0.4134   FAQ                 FAQ: How do I reset my password?
2   DOC011   0.2176   Support/Incidents   Ticket: User locked out after failed password attempts
3   DOC003   0.2032   Policy/Process      VPN and Remote Work Policy
4   DOC022   0.1879   FAQ                 FAQ: How do I connect to the VPN?
5   DOC002   0.1862   Policy/Process      User Onboarding and Access Assignment Process

[Output saved to evidencias\test1_policy_process.txt]
```

Figure 2

Test 2 (Support/Incidents): recommendations for seed document DOC012

```
(.venv) C:\Users\Admin\Downloads\document-recommender\document-recommender>python recommender.py --id DOC012
================================================================================================
INPUT (ID): DOC012 | Support/Incidents | Ticket: VPN will not connect from home, authentication error
================================================================================================
Top-5 recommended documents (cosine similarity)
#   ID       Score    Category            Title
------------------------------------------------------------------------------------------------
1   DOC022   0.5002   FAQ                 FAQ: How do I connect to the VPN?
2   DOC003   0.3073   Policy/Process      VPN and Remote Work Policy
3   DOC001   0.1545   Policy/Process      Password and Multi-Factor Authentication Policy
4   DOC011   0.1417   Support/Incidents   Ticket: User locked out after failed password attempts
5   DOC021   0.1304   FAQ                 FAQ: How do I reset my password?
```

Figure 3

Test 3 (free text): recommendations for the query "My account is locked after I changed my password"

```
(.venv) C:\Users\Admin\Downloads\document-recommender\document-recommender>python recommender.py --query "My account is locked after I changed my password"
================================================================================================
INPUT (TEXT): My account is locked after I changed my password
================================================================================================
Top-5 recommended documents (cosine similarity)
#   ID       Score    Category            Title
------------------------------------------------------------------------------------------------
1   DOC011   0.4455   Support/Incidents   Ticket: User locked out after failed password attempts
2   DOC001   0.2713   Policy/Process      Password and Multi-Factor Authentication Policy
3   DOC021   0.2293   FAQ                 FAQ: How do I reset my password?
4   DOC009   0.2105   Policy/Process      Employee Offboarding and Access Revocation Process
5   DOC025   0.1838   FAQ                 FAQ: What should I do if I lose my company laptop or phone?
```

Figure 4

Top-5 category coherence obtained with the --evaluate option

```
(.venv) C:\Users\Admin\Downloads\document-recommender\document-recommender>python recommender.py --evaluate
Category coherence (fraction of the Top-5 that shares the seed's category)
------------------------------------------------------------------------------------------------
Policy/Process         documents: 10  average: 0.38
Support/Incidents      documents: 10  average: 0.30
FAQ                    documents: 5   average: 0.24
Minutes                documents: 5   average: 0.40
OVERALL                documents: 30  average: 0.33
```